

Congestion-Aware Scalable Service Migration in Resource-Constrained Wide Area Networks

Yu Jiang, *Member, IEEE*, Wei Feng, *Senior Member, IEEE*, Ning Ge, *Member, IEEE*

Abstract—Under the mobile edge computing (MEC) paradigm, user mobility necessitates service migration across the backhaul network to guarantee delay performance. However, in resource-constrained wide area networks (WANs) with satellite and UAV links, service migration faces a fundamental trade-off between user delay and backhaul link congestion. Besides, the joint migration decision and resource allocation problem is a mixed-integer nonlinear programming (MINLP) with prohibitively high complexity, leading to severe scalability challenges. To address these issues, this work proposes the Congestion-Aware Scalable Migration (CASM) algorithm. Unlike conventional congestion control schemes focusing on path selection, CASM jointly optimizes migration decision, path selection, and data rate allocation under a congestion-aware architecture. First, the original MINLP problem is decomposed via a two-layer framework, where a novel subnet partitioning method is introduced to enable per-node decomposition of link capacity constraints. Then, a congestion-aware Markov Decision Process (MDP) is designed based on the fine-grained subproblem, where a well-designed reward function is adopted to balance the delay-congestion trade-off. Experimental results validate that the proposed two-layer decomposition endows CASM with favorable scalability. Compared with existing approaches, CASM achieves up to 25% delay reduction via improved link resource utilization, and its performance gain is more pronounced in resource-constrained and low-connectivity networks.

Index Terms—Service Migration, Problem Decomposition, Markov Decision Process, Congestion Control

I. INTRODUCTION

The proliferation of mobile edge computing (MEC) has reshaped the management of compute-intensive services in modern wide area networks (WANs) [1]. Under the MEC paradigm, services are offloaded to edge servers deployed at access nodes (ANs), ranging from terrestrial base stations to satellites and UAVs for remote and emergency use cases [2]. This framework supports diverse delay-sensitive applications, including vehicular networking, IoT-driven sensing, and emerging artificial intelligence (AI) services such as large language model (LLM)-based personal agents [3]. Enabling such service provisioning, Software-Defined Networking (SDN) technologies provide a unified framework for flexible network management and service orchestration [4].

Considering that user mobility increases the delay between a user and its remotely hosted service instance, maintaining

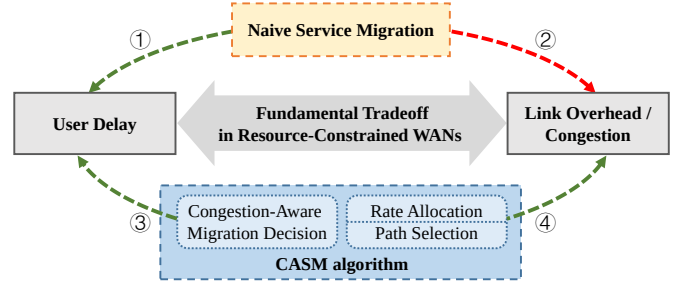


Fig. 1. An illustration of the fundamental trade-off between QoS/delay and backhaul link overhead/congestion (arrow 1, 2), and the effects of our CASM algorithm on balancing both (arrow 3, 4).

acceptable quality of service (QoS) requires dynamic service relocation, a process known as service migration [5], which encompasses two key issues: (1) Migration Decision (MD), i.e., determining whether and to which AN each service instance should be migrated; (2) Resource Allocation (RA), including both node RA, which allocates computation and communication resources at each AN, and link RA, which allocates data rates for each migrating service on each backhaul link. Note that MD and RA are tightly coupled, with RA relying on determined migration decisions for each service instance.

A. Trade-off and Challenges

While service migration reduces access delay for mobile users, it incurs substantial backhaul overhead in resource-constrained WANs with heterogeneous infrastructures, where wired fiber links coexist with satellite or UAV links. For instance, low-Earth orbit (LEO) satellite-to-ground links typically support only 0.1-1 Gbps, orders of magnitude lower than terrestrial fiber links, which usually reach 100 Gbps - 1 Tbps [6]. Compounding this limitation is the rapidly growing size of service instances driven by advances in AI applications: the weight parameters of an LLM can occupy tens of gigabytes [7], making the migration data size far larger than the backhaul communication data. Therefore, as shown in Fig. 1, when massive service instances migrate simultaneously, naive migration strategies can maintain individual QoS by reducing access delay (arrow 1), yet induce excessive overhead and severe congestion to the backhaul network (arrow 2). Such congestion may trigger migration failures and paradoxically degrade overall QoS [8]. Thus, an optimization framework is required to balance the fundamental trade-off between user delay and backhaul link overhead/congestion, so as to achieve congestion alleviation while maintaining satisfactory overall

This work was supported in part by the National Natural Science Foundation of China under Grant 62425110, Grant 62541104, and Grant U22A2002, in part by the Suzhou Science and Technology Project, and in part by the FAW Jiefang Automotive Co., Ltd. (Corresponding author: Ning Ge.)

The authors are with the Department of Electronic Engineering, Tsinghua University, Beijing 100084, China (e-mail: jiangy20@mails.tsinghua.edu.cn; fengwei@tsinghua.edu.cn; gening@tsinghua.edu.cn).

QoS. Generally, such optimization faces two challenges in resource-constrained WANs.

(1) Extensive scale and decomposition difficulty

For a backhaul network with V ANs and N users, the joint MD and RA problem is a mixed-integer nonlinear program (MINLP) whose feasible space grows exponentially with N and V , exhibiting the curse of dimensionality. Directly modeling it as an MDP yields a state space of size V^{2N} [9], which is computationally intractable at practical scales. Although existing works (e.g., [9] [10] [11]) enable per-node decomposition by ignoring link capacity constraints, such constraints are essential in resource-constrained backhaul networks and render decomposition much harder. Specifically, link capacity constraints are hard to be decomposed by ANs, since each backhaul link may carry migration traffic toward different ANs, thereby coupling migration decisions via shared link capacity constraints.

(2) Link Congestion Control

To alleviate link congestion caused by service migration, most existing works (e.g., [12] [13] [14]) optimize path selection with flexible routing to bypass congested links, while assuming fixed data rates for each migrating service. However, in scenarios such as remote regions and emergency communications, network topologies often exhibit low connectivity. Consequently, the lack of alternative paths fundamentally limits the effectiveness of congestion control schemes purely based on path selection. In such contexts, incorporating data rate optimization is a promising way to improve the upper bound of congestion alleviation performance. Such fine-grained rate allocation in WANs is enabled by SDN technologies. A representative implementation is CN-WAN [15], a system that dynamically throttles traffic according to service type, application ID, and user priority.

B. Our Work

This paper presents a congestion-aware scalable approach for joint MD and RA in resource-constrained WANs. First, taking the total delay as the objective, we formulate the original problem as an MINLP. To address the first challenge, we decompose the original problem through a two-layer framework. For node RA, direct decomposition yields per-AN convex subproblems which can be solved with off-the-shelf solvers. For MD and link RA, dual decomposition further decomposes the problem into a set of finest-grained subproblems, each optimizing the migration destination and data rate for a single user. The objective combines delay with node and link overheads introduced via Lagrangian duality, where the Lagrange multipliers serve as resource prices for cross-layer coordination. Specifically, to enable per-AN decomposition of link capacity constraints, we propose a subnet partitioning method that divides the resource capacity of each backhaul link into V parts, corresponding to V virtual subnets, each dedicated to carrying migration traffic toward a specific destination AN. The proposed decomposition framework eliminates the curse of dimensionality, reducing the state space from V^{2N} to $N_v \cdot V^2$, where N_v denotes the number of users under a single AN.

To address the second challenge, we propose the Congestion-Aware Scalable Migration (CASM) algorithm, where the

fine-grained per-user subproblem is cast as an MDP with a congestion-aware reward function constructed from its objective. Additionally, CASM integrates a load-estimation-based subnet resource allocation scheme, and a heuristic rate-throttling mechanism to prevent capacity constraint violations arising from subnet allocation inaccuracies. Our algorithm acquires congestion awareness through link weight updates that reflect backhaul congestion levels, thus unifying MD, path selection, and data rate allocation in a single framework. By augmenting path selection with data rate allocation, CASM can reduce user delay and elevate congestion control performance, a benefit particularly marked in low-connectivity networks.

The main contributions of this work are listed as follows:

- An online optimization model is developed, where we formulate an MINLP problem for joint MD and RA, considering resource capacity constraints on both ANs and backhaul links.
- A two-layer decomposition framework is proposed to reduce problem complexity and enable parallel optimization. In particular, a virtual subnet partitioning method is introduced to enable per-AN decomposition of link capacity constraints. To our knowledge, this is the first application of subnet partitioning to service migration problems.
- A congestion-aware iterative algorithm is designed, where the link weights are dynamically updated to guide migration decision, data rate allocation, and path selection. To our knowledge, this is the first work to jointly optimize these three aspects within a unified congestion-aware framework.
- Extensive experiments have validated the delay-congestion trade-off. Compared with path-selection-based baselines, our approach exhibits superior capability in congestion alleviation and achieves lower user delay, particularly in network topologies with low connectivity.

The rest of this paper is organized as follows. The related work is reviewed in Section II. Section III presents the system model and formulates the original problem, which is decomposed in Section IV with the proposed two-layer framework. In Section V, we illustrate the CASM algorithm, and simulation results are evaluated in Section VI.

II. RELATED WORK

A. Direct Approaches

In the recent literature, machine learning (ML), particularly deep reinforcement learning (DRL), has been widely applied to service migration problems to cope with the dynamic and uncertain nature of user mobility and network conditions [16]. In vehicular edge computing (VEC) scenarios, Yuan et al. [17] proposed a multi-agent DRL model with a two-branch convolutional deep Q-network to jointly optimize service migration and vehicle routing. Peng et al. [18] introduced a fast transfer reinforcement learning (TL) method for communication cost-aware migration. Gao et al. [19] and Khamari et al. [20] explored decentralized and adaptive learning strategies to address scalability and rapidly changing vehicular conditions. Chen et al. [21] applied DRL to multi-task migration scheduling

to cope with heterogeneous demands in VEC. In general MEC settings, Wang et al. [22] modeled microservice coordination as an MDP and proposed a near-optimal online DRL algorithm. Wang et al. [23] extended this to a partially observable MDP, designing an actor-critic framework with LSTM-based hidden-state modeling. As for the related problem of computation offloading, Ale et al. [24] and Zhou et al. [25] adopted DRL in dynamic multi-server MEC, designing an energy-efficient migration framework under delay constraints.

Despite their effectiveness, directly applying ML without problem decomposition faces two major obstacles. First, the action space for joint MD and RA grows exponentially, hampering efficient exploration and sample utilization. Second, capacity constraints on heterogeneous resources are frequently violated during the learning process, leading to sample waste and low training efficiency. These limitations underscore the need for problem decomposition, as detailed below.

B. Decomposition Methods

To overcome the limitations of direct approaches, a rich body of work applies problem decomposition prior to resolution. Lyapunov optimization transforms long-term stochastic optimization into deterministic per-slot sub-problems via virtual queue tracking, achieving provable cost-constraint tradeoffs without prior knowledge of future system states. Adopting this technique, Ouyang et al. [26] decomposed MEC service placement under long-term cost budgets into real-time subproblems solvable without mobility prediction. Labriji et al. [27] integrated 5G handover-based mobility estimation to derive a closed-form IoV migration algorithm for dense urban scenarios. Chen et al. [28] optimized microservice migration and used randomized rounding to tackle per-slot NP-hard subproblems. Wang et al. [29] and Zhou et al. [30] embedded mobility prediction into Lyapunov frameworks for proactive cost reduction. Huang et al. [31] extended the method to MEC emergency and failure recovery. Sun et al. [32] proposed the EMM algorithm for ultra-dense MEC, combining Lyapunov optimization and multi-armed bandit to allocate network resources under energy constraints. Similarly, Zhou et al. [33] developed a scheme using Lyapunov and particle swarm optimization to solve non-convex per-slot subproblems under multi-user interference.

Beyond Lyapunov-based methods, block coordinate descent (BCD) and primal/dual decomposition provide a complementary approach. The seminal work of Chiang et al. [34] established a mathematical foundation for understanding layered network architectures as distributed solutions to network utility maximization problems. The paper developed both primal and dual decomposition, where subproblems are coordinated through Lagrange multipliers. This framework was adopted in [9] to solve the service migration problem in multi-user satellite MEC, where the curse of dimensionality was eliminated by solving the fine-grained sub-problems for each user. Tran [10] and Liu [11] represent further BCD-based contributions, decomposing joint service offloading and resource allocation problems into tractable sub-problems with closed-form and approximation-algorithm solutions, respectively.

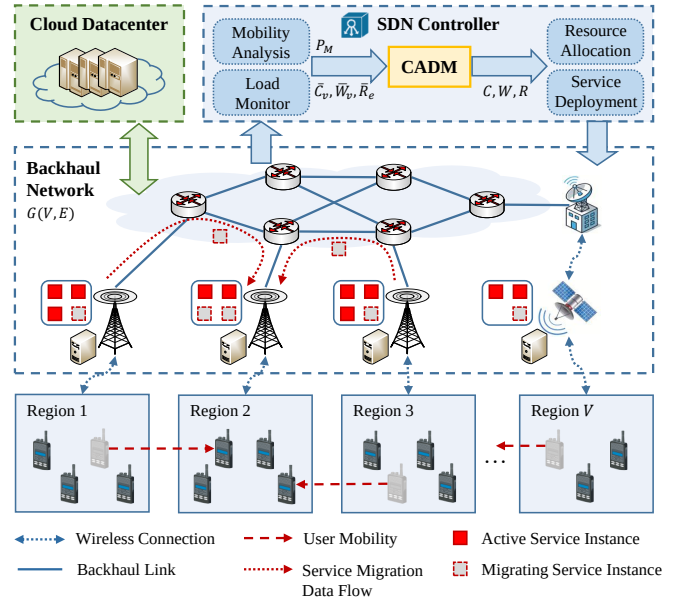


Fig. 2. An illustration of the system model, with service migrations through a bandwidth-constrained backhaul network.

C. Congestion-Aware Methods

While the works reviewed above have made significant advances in minimizing user delay and migration cost, a common limitation persists: the majority of studies merely characterize the backhaul network by aggregating migration cost into the objective function, or taking the total migration cost as a constraint, without enforcing hard capacity constraints on each backhaul link. As mentioned in Section I, unregulated link utilization can lead to severe link congestion and service degradation. Several works address link congestion through path selection and traffic steering. Xu et al. [12] formulated VEC path selection as a Cost-QoE multi-objective problem and designed a Pareto-optimal filtering method. Anwar et al. [13] presented a distributed traffic steering scheme, adopting network partitioning and dynamic shortest-path selection to mitigate congestion. Cao et al. [35] employed a DQN agent to select low-cost, low-delay migration paths while avoiding congested links. Liu et al. [36] jointly optimized VNF placement and routing to alleviate link congestion under fluctuating user mobility. Jasim et al. [37] introduced asynchronous coarse-grained load migration for NFV-enabled fog networks to prevent node saturation. Xiao et al. [14] considered constrained backhaul bandwidth in VEC cell networks and devised the Lyapunov-based algorithm to guarantee QoS under limited link capacity.

While the above methods represent important progress, they primarily address network congestion through path selection, while the data rates for each migrating service are typically treated as fixed quantities. In topologies with limited path diversity, path selection alone provides insufficient congestion relief, as few disjoint paths exist between source and destination nodes. In such scenarios, jointly optimizing migration data rate allocation alongside path selection can yield a richer solution space. To the best of our knowledge, this work is the first to jointly optimize migration decision, data rate allocation, and path selection within a unified congestion-aware framework.

III. PROBLEM FORMULATION

A. System Model

Fig. 2 shows the system architecture for service migration in WANs. First, the backhaul network is modeled as an undirected graph $G(V, E)$, where the node set contains several switches and V ANs, each equipped with an MEC server and covering a fixed region. For each backhaul link $e \in E$, the communication resource capacity is given by its maximum supported data rate $\bar{R}_e$. For each AN $v \in V$, the computation and bandwidth capacities are denoted as $\bar{C}_v$ and $\bar{W}_v$, respectively. Let N denote the set of mobile users, and u_n denotes the user location. Each user is associated with a personalized agent, which is implemented as a service instance hosted on an AN. Thus, the number of users equals that of service instances. A user accesses the AN covering its current region and offloads tasks to the AN hosting its service instance. If these two ANs differ, the user request is forwarded via backhaul network G . Then the service instance processes the request and returns the result.

The considered system runs in discrete time slots, denoted as $t = 1, 2, \dots, T$. During each time slot t , a user may move into the coverage region of another AN, potentially necessitating the migration of its service instance. The user mobility is modeled as a transition probability matrix $P_M \in \mathbb{R}^{V \times V}$, where $P_M(u_n^{t-1}, u_n^t) \in [0, 1]$ denotes the probability that a user moves to location u_n^t at time slot t . For each user $n \in N$, its agent service is characterized by a three-tuple $SV_n = (\phi_n^t, D_n^t, I_n^t)$, where ϕ_n^t denotes the required computational task size (in CPU cycles) during time slot t , D_n^t represents the remote communication data size when user n and SV_n are not co-located at the same AN, and I_n^t is the data size to be transferred through G if SV_n is migrated.

B. Optimization Formulation

The set of optimization variables is denoted as (C, W, R) , where C and W correspond to node RA, representing the allocation of computation and communication resources on each AN, while R includes both the migration destination and the allocated data rate for transmitting I_n^t , corresponding to MD and link RA, respectively. Detailed definitions are listed as follows.

Variables

- C_n^t : continuous scalar, computation resource allocated to user n at time slot t .
- W_n^t : continuous scalar, bandwidth resource allocated to user n at time slot t .
- $R_n^t \in \mathbb{R}^{V \times 1}$: continuous vector, $R_n^t(v) \neq 0$ denotes that SV_n deploys on AN v at time t , and the data rate for user n 's backhaul communications is $\sum_{v \in V} R_n^t(v)$. Given R_n^t , the deployment location of SV_n at time slot t can be derived as $x_n^t = h(R_n^t)$, where function $h(a)$ means finding the position of non-zero element in vector a .

Objective

The optimization objective is defined as the weighted sum of the total delay for each user $n \in N$, denoted as $\xi_n T_n$, where the weight ξ_n is taken as the priority of user n . The total delay

T_n consists of three components: computation, communication, and migration delays, as detailed below.

$$\min_{C, W, R} \sum_{n \in N} \xi_n \cdot (T_n^s + T_n^c + T_n^m) \quad (1)$$

First, the computation delay of user n is given by the ratio of its computational task size to the allocated computing resource, denoted as follows:

$$T_n^s = \sum_{t=1}^T \frac{\phi_n^t}{C_n^t} \quad (2)$$

Second, the communication delay of user n includes the local access delay T_n^a and the backhaul delay T_n^b :

$$T_n^c = \sum_{t=1}^T (T_n^a(t) + T_n^b(t)) \quad (3)$$

$$T_n^a(t) = \frac{m_n \cdot D_n^t}{W_n^t \cdot \rho(u_n^t)} \quad (3a)$$

$$T_n^b(t) = I_{\{x_n^t \neq u_n^t\}} \cdot \frac{m_n \cdot D_n^t}{\sum_{v \in V} R_n^t(v)} \quad (3b)$$

where m_n denotes the number of communication rounds between user n and SV_n per time slot, $\rho(u_n^t) = \log_2(1 + \text{SNR}(u_n^t))$ represents the channel condition at user n 's current location, and $I\{a\}$ is an indicator function that equals 1 if the condition a is true and 0 otherwise. (3b) means that backhaul communication takes place only when $x_n^t \neq u_n^t$.

Third, the migration delay T_n^m occurs only when SV_n is under migration during the current time slot (i.e., $x_n^t \neq x_n^{t-1}$), and is given by:

$$T_n^m = \sum_{t=1}^T \left[I_{\{x_n^t \neq x_n^{t-1}\}} \cdot \left(t_n^d + \frac{I_n^t}{\sum_{v \in V} R_n^t(v)} \right) \right] \quad (4)$$

where the constant t_n^d denotes the service downtime, including the shutdown time at the source AN x_n^{t-1} and the startup time at the destination x_n^t .

Constraints

$$C_n^l \leq C_n^t \leq C_n^u \quad \forall n \in N, \forall t \in T \quad (5)$$

$$W_n^l \leq W_n^t \leq W_n^u \quad \forall n \in N, \forall t \in T \quad (6)$$

$$R_n^t(v) \in [R_n^l, R_n^u] \cup \{0\} \quad \forall v \in V, \forall n \in N, \forall t \in T \quad (7)$$

$$\sum_{v \in V} I_{\{R_n^t(v) > 0\}} = 1 \quad \forall n \in N, \forall t \in T \quad (8)$$

$$\sum_{n \in N} I_{\{x_n^t = v\}} C_n^t \leq \bar{C}_v \quad \forall v \in V, \forall t \in T \quad (9)$$

$$\sum_{n \in N} I_{\{x_n^t = v\}} W_n^t \leq \bar{W}_v \quad \forall v \in V, \forall t \in T \quad (10)$$

$$L_e^t \leq \bar{R}_e \quad \forall e \in E, \forall t \in T \quad (11)$$

Constraints (5) - (7) specify the feasible ranges of C, W, R , respectively, where each variable is assigned an upper bound and a lower bound. (8) ensures the uniqueness of the service migration destination. (9) and (10) correspond to node RA, ensuring that the total computing and bandwidth resources occupied on each AN do not exceed its capacity. While (11) are link capacity constraints related to link RA, requiring that

the aggregate load L_e^t on each backhaul link $e \in E$ does not exceed its capacity $\bar{R}_e$. Specifically, to derive L_e^t , the concepts of Source-Destination-Pair (SDP) i and routing matrix P are introduced. First, we define a mapping $i = \Pi(v_1, v_2)$ that identifies the SDP with given source v_1 and destination v_2 , and let I_G denote the set of all SDPs in network G , where obviously $|I_G| = V^2$. Second, to characterize the routing rules, we introduce a binary matrix $P \in \{0, 1\}^{V^2 \times E}$, where $P(i, e) = 1$ only when link e lies on the path designated for SDP i . With the above definitions, the link load L_e^t is derived as follows.

$$L_e^t = \sum_{n \in N} \sum_{v \in V} R_n^t(v) P(i_n^t, e), \quad i_n^t = \Pi(x_n^{t-1}, x_n^t) \quad (12)$$

Based on the definitions and derivations above, the original MINLP problem $\mathbf{P}_1(\mathbf{C}, \mathbf{W}, \mathbf{R})$ is summarized as below.

$$\begin{aligned} \min_{C, W, R} \quad & \sum_{n \in N} \xi_n \cdot (T_n^s + T_n^c + T_n^m) \\ \text{s.t.} \quad & (5) - (12) \end{aligned} \quad (13)$$

Jointly optimizing MD and RA, $\mathbf{P}_1$ can be reformulated as a classic knapsack problem, which has been proven to be NP-hard [21]. The introduction of the migration variable R significantly increases the problem complexity. If an MDP model is directly constructed based on $\mathbf{P}_1$ to solve the service migration problem for all users $n \in N$, the resulting state space would be extremely high-dimensional. For instance, defining the state at time slot t as $S^t = (x_1^t, \dots, x_N^t; u_1^t, \dots, u_N^t)$ yields $|S^t| = V^{2N}$. The exponentially growing state space leads to the curse of dimensionality, making it infeasible for traditional policy iteration or Q-learning methods to solve $\mathbf{P}_1$ in practical-scale scenarios. Moreover, as discussed in Section II, directly applying ML methods (e.g., DQN, DDPG, PPO) to solve $\mathbf{P}_1$ also faces severe challenges, including inefficient exploration, unstable training, and frequent constraint violations during learning. Therefore, a series of decompositions is required to transform $\mathbf{P}_1$ into low-complexity and fine-grained subproblems, which can be cast as MDPs with smaller and feasible state spaces.

IV. PROBLEM DECOMPOSITION

As illustrated in Fig. 3, a two-layer framework is proposed in this section, which consists of two types of decomposition, as shown in the red dashed boxes.

- **Direct decomposition:** First, variable decomposition is applied to $\mathbf{P}_1$, decoupling node RA from MD and link RA, yielding $\mathbf{P}_1(\mathbf{R})$ for MD & link RA, and $\mathbf{P}_1(\mathbf{C}, \mathbf{W})$ for node RA. Then, for each AN v , we perform AN decomposition to obtain two sets of subproblems: the node RA subproblems $\mathbf{P}_2$ and the service migration & link RA subproblems $\mathbf{P}_3$.
- **Dual decomposition:** Lagrange duality is applied to $\mathbf{P}_3$ to obtain its dual problem $\mathbf{P}_4$, which is then further decomposed for each user, resulting in the finest-grained subproblem $\mathbf{P}_5$, i.e., the MD & link RA subproblem for each single user.

A detailed elaboration is given below.

A. Direct Decomposition

First, $\mathbf{P}_1(\mathbf{C}, \mathbf{W}, \mathbf{R})$ is transformed into: (a) $\mathbf{P}_1(\mathbf{C}, \mathbf{W})$ with $\mathbf{R}$ fixed, corresponding to node RA; (b) $\mathbf{P}_1(\mathbf{R})$ with $\mathbf{C}, \mathbf{W}$ fixed, corresponding to MD and link RA. The objective and constraints of these two subproblems are identical to those of (13), and are therefore omitted. In $\mathbf{P}_1(\mathbf{C}, \mathbf{W})$, since $\mathbf{R}$ is fixed, the objectives and constraints of node RA under each AN are mutually independent. Therefore, it can be decomposed for each AN $v \in V$, producing a total of V subproblems, as detailed below.

$$\mathbf{P}_2(\mathbf{C}, \mathbf{W}) : \min_{C, W} \sum_{n \in N_v} \xi_n \cdot (T_n^s + T_n^c + T_n^m) \quad (14)$$

$$\text{s.t.} \quad \begin{cases} C_n^l \leq C_n^t \leq C_n^u & \forall n \in N, \forall t \in T \\ W_n^l \leq W_n^t \leq W_n^u & \forall n \in N, \forall t \in T \\ \sum_{n \in N_v} C_n^t \leq \bar{C}_v & \forall t \in T \\ \sum_{n \in N_v} W_n^t \leq \bar{W}_v & \forall t \in T \end{cases} \quad (15)$$

Here we define the user subset $N_v = \{n \in N | x_n^t = v\}$, meaning that the whole user set N is partitioned according to current migration destinations $x_n^t = h(R_n^t), \forall n \in N$.

Furthermore, as the allocation of computation (variable C) and communication (variable W) resources are mutually independent in each AN, $\mathbf{P}_2(\mathbf{C}, \mathbf{W})$ can be further decomposed into $\mathbf{P}_2(\mathbf{C})$ and $\mathbf{P}_2(\mathbf{W})$ as follows.

$$\mathbf{P}_2(\mathbf{C}) : \min_C \sum_{n \in N_v} \xi_n \cdot T_n^s \quad (16)$$

$$\text{s.t.} \quad \begin{cases} C_n^l \leq C_n^t \leq C_n^u & \forall n \in N, \forall t \in T \\ \sum_{n \in N_v} C_n^t \leq \bar{C}_v & \forall t \in T \end{cases} \quad (17)$$

$$\mathbf{P}_2(\mathbf{W}) : \min_W \sum_{n \in N_v} \xi_n \cdot T_n^a \quad (18)$$

$$\text{s.t.} \quad \begin{cases} W_n^l \leq W_n^t \leq W_n^u & \forall n \in N, \forall t \in T \\ \sum_{n \in N_v} W_n^t \leq \bar{W}_v & \forall t \in T \end{cases} \quad (19)$$

It can be proven that $\mathbf{P}_2(\mathbf{C}, \mathbf{W})$ is a convex optimization problem [21], which can thus be solved directly using off-the-shelf convex optimization solvers (e.g., CPLEX, Gurobi). In particular, for each user $n \in N_v$, if the feasible range constraints in (17) and (19) are relaxed to $C_n^l = 0^+$, $C_n^u = \bar{C}_v$ and $W_n^l = 0^+$, $W_n^u = \bar{W}_v$, the optimal solutions for C and W can be derived in closed form, which are proportional to the product of user weight ξ_n and its resource demand ϕ_n^t or ψ_n^t , as shown below.

$$C_n^{t*} = \frac{\sqrt{\xi_n \phi_n^t} \cdot \bar{C}_v}{\sum_{n \in N_v} \sqrt{\xi_n \phi_n^t}} \quad (20)$$

$$W_n^{t*} = \frac{\sqrt{\xi_n \psi_n^t} \cdot \bar{W}_v}{\sum_{n \in N_v} \sqrt{\xi_n \psi_n^t}}, \quad \psi_n^t = \frac{m_n D_n^t}{\rho(u_n^t)} \quad (21)$$

B. Virtual Subnet Partition

Different from $\mathbf{P}_1(\mathbf{C}, \mathbf{W})$, AN decomposition cannot be directly applied to $\mathbf{P}_1(\mathbf{R})$ due to the presence of link capacity constraints (11). For each link $e \in E$, its load L_e^t may originate from any migrating user service across network G , whose migration destinations may vary. Considering such coupling

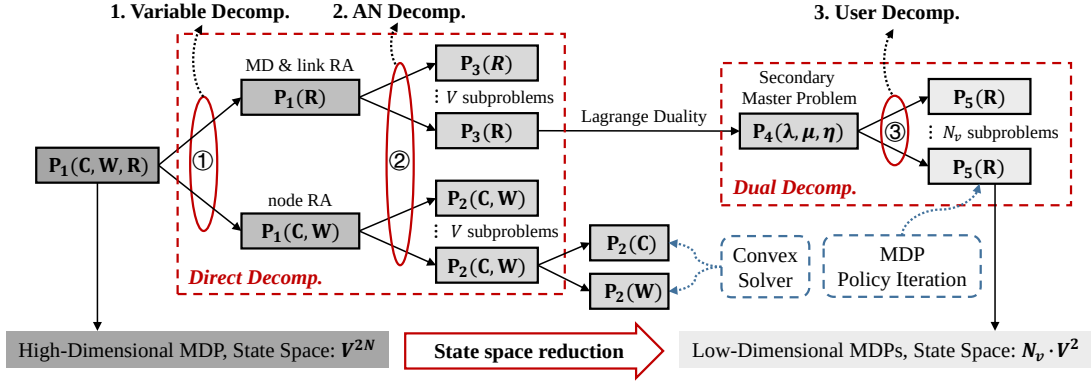


Fig. 3. Detailed two-layer decomposition framework, including the direct and dual decompositions. $P_2(C)$ and $P_2(W)$ are convex optimization problems and can be solved by off-the-shelf convex solvers, while $P_5(R)$ is transformed into an MDP and solved by policy iteration. Compared to the original problem $P_1(C, W, R)$, the MDP problem transferred from $P_5(R)$ exhibits a substantially smaller state space (from V^{2N} to $N_v \cdot V^2$).

between ANs and backhaul links, without knowledge of the migration decisions R , constraints (11) cannot be directly decomposed by AN.

To address this issue, the concept of virtual subnet is introduced, where we assign a virtual subnet G_v to each user subset N_v , dedicated to transmitting service migration and backhaul communication data for users $n \in N_v$. Furthermore, for each backhaul link $e \in E$, its resources need to be partitioned based on the loads from each user subset N_v . However, since the exact migration decision is unknown at this stage, it is necessary to estimate these loads, and such estimation is detailed in Section V-B. After virtual subnet partitioning, the resource capacity of each backhaul link e is divided into V parts, denoted as follows.

$$\bar{R}_e = \sum_{v \in V} \bar{S}R_{e,v}^t \quad (22)$$

where $\bar{S}R_{e,v}^t$ is the link resource capacity assigned to G_v at time slot t . Consequently, the link capacity constraints in (11) are modified as follows.

$$L_{e,v}^t \leq \bar{S}R_{e,v}^t \quad \forall v \in V, \forall e \in E, \forall t \in T \quad (23)$$

$$L_{e,v}^t = \sum_{n \in N_v} R_n^t(v) P(i_n^t, e), \quad i_n^t = \Pi(x_n^{t-1}, x_n^t) \quad (24)$$

where $L_{e,v}^t$ denotes the load on link e from user subset N_v at time t . Compared with (11), the number of link capacity constraints expands from E to $E \times V$ for each time slot, and the node-link coupling in (11) is eliminated, thus enabling per-AN decomposition of $P_1(R)$. The subproblem on each AN v is summarized as follows.

$$P_3(R) : \min_R \sum_{n \in N_v} \xi_n \cdot (T_n^s + T_n^c + T_n^m) \quad (25)$$

$$s.t. \begin{cases} R_n^t(v) \in [R_n^l, R_n^u] \cup \{0\} & \forall n \in N, \forall t \in T \\ \sum_{v \in V} I_{\{R_n^t(v) > 0\}} = 1 & \forall n \in N, \forall t \in T \\ \sum_{n \in N_v} C_n^t \leq \bar{C}_v & \forall t \in T \\ \sum_{n \in N_v} W_n^t \leq \bar{W}_v & \forall t \in T \\ L_{e,v}^t \leq \bar{S}R_{e,v}^t & \forall e \in E, \forall t \in T \end{cases} \quad (26)$$

The formulation of $P_2(C, W)$ and $P_3(R)$ belongs to direct decomposition, which not only reduces complexity but also

enables parallel solving for each AN. Next, dual decomposition is applied to $P_3(R)$ for further complexity reduction.

C. Dual Decomposition

The core idea of dual decomposition is to incorporate coupling constraints into the objective function via Lagrange multipliers (i.e., prices for heterogeneous resources on each node and link), thereby decomposing the original problem into multiple independent subproblems and coordinating these prices to achieve global optimality [34]. First, the Lagrange function of P_3 's objective (25) is derived as follows.

$$\begin{aligned} L(R, \lambda, \mu, \bar{\eta}) = & \sum_{n \in N_v} \xi_n \cdot (T_n^s + T_n^c + T_n^m) \\ & + \sum_{t=1}^T \lambda^t \cdot \left(\sum_{n \in N_v} C_n^t - \bar{C}_v \right) + \sum_{t=1}^T \mu^t \cdot \left(\sum_{n \in N_v} W_n^t - \bar{W}_v \right) \\ & + \sum_{t=1}^T \sum_{e \in E} \eta_e^t \cdot \left(\sum_{n \in N_v} R_n^t(v) P(i_n^t, e) - \bar{R}_e^t(v) \right) \end{aligned} \quad (27)$$

where $i_n^t = \Pi(x_n^{t-1}, x_n^t)$, and $\lambda^t, \mu^t, \bar{\eta}$ are Lagrange multipliers. λ^t, μ^t represent the unit prices of computational and bandwidth resources on AN v , respectively, while $\bar{\eta} = \{\eta_e^t | e \in E\}$ denotes the unit resource price on each link e in subnet G_v for users $n \in N_v$. Then, the following transformation is applied to (27) by moving the summation $\sum_{n \in N_v}$ forward.

$$\begin{aligned} L(R, \lambda, \mu, \bar{\eta}) = & \sum_{n \in N_v} \left[\xi_n (T_n^s + T_n^c + T_n^m) + \sum_{t=1}^T \lambda^t \cdot C_n^t \right. \\ & + \sum_{t=1}^T \mu^t \cdot W_n^t + \sum_{t=1}^T \sum_{e \in E} \eta_e^t \cdot R_n^t(v) P(i_n^t, e) \left. \right] \\ & - \left[\sum_{t=1}^T \left(\lambda^t \cdot \bar{C}_v + \mu^t \cdot \bar{W}_v + \sum_{e \in E} \eta_e^t \cdot \bar{R}_e^t(v) \right) \right] \\ = & \sum_{n \in N_v} g_n(R, \lambda, \mu, \bar{\eta}) - K(\lambda, \mu, \bar{\eta}) \end{aligned} \quad (28)$$

$$\begin{aligned}
g_n(R, \lambda, \mu, \vec{\eta}) &= \xi_n(T_n^s + T_n^c + T_n^m) + \sum_{t=1}^T \lambda^t \cdot C_n^t \\
&+ \sum_{t=1}^T \mu^t \cdot W_n^t + \sum_{t=1}^T \sum_{e \in E} \eta_e^t \cdot R_n^t(v) P(i_n^t, e) \\
K(\lambda, \mu, \vec{\eta}) &= \sum_{t=1}^T \left(\lambda^t \cdot \bar{C}_v + \mu^t \cdot \bar{W}_v + \sum_{e \in E} \eta_e^t \cdot \bar{R}_e^t(v) \right)
\end{aligned} \quad (29)$$

With the above transformation, the dual problem $\mathbf{P}_4(\mathbf{R})$ is obtained as follows, serving as the secondary master problem.

$$\mathbf{P}_4(\mathbf{R}) : \max_{\lambda, \mu, \vec{\eta}} \sum_{n \in N_v} g_n(\lambda, \mu, \vec{\eta}) - K(\lambda, \mu, \vec{\eta}) \quad (31)$$

$$s.t. \quad \lambda \succeq 0, \mu \succeq 0, \vec{\eta} \succeq 0 \quad (32)$$

Subsequently, user decomposition is applied to $\mathbf{P}_4(\mathbf{R})$, yielding a set of the finest-grained subproblems for per-user service migration, denoted as $\mathbf{P}_5(\mathbf{R})$. Note that $K(\lambda, \mu, \vec{\eta})$ (30) is independent of variable R , the optimization objective reduces to only $g_n(R, \lambda, \mu, \vec{\eta})$ (29).

$$\mathbf{P}_5(\mathbf{R}) : \min_R g_n(R, \lambda, \mu, \vec{\eta}) \quad (33)$$

$$s.t. \quad \begin{cases} R_n^t(v) \in [R_n^l, R_n^u] \cup \{0\} & \forall t \in T \\ \sum_{v \in V} I_{\{R_n^t(v) > 0\}} = 1 & \forall t \in T \end{cases} \quad (34)$$

$\mathbf{P}_5$ focuses on the service migration of a single user n , and g_n consists of two parts: the original objective, i.e., the total delay T_n , and two overheads introduced by Lagrangian duality, namely the Node Overhead NO_n and Link Overhead LO_n . Therefore, g_n (33) can be rewritten as follows.

$$g_n = \xi_n T_n + NO_n + LO_n \quad (35)$$

$$NO_n = \sum_{t=1}^T \lambda^t \cdot C_n^t + \sum_{t=1}^T \mu^t \cdot W_n^t \quad (36)$$

$$LO_n = \sum_{t=1}^T \sum_{e \in E} \eta_e^t \cdot R_n^t(v) P(i_n^t, e) \quad (37)$$

Compared with $\mathbf{P}_3$, which only optimizes user delay, $\mathbf{P}_5$ incorporates resource overheads into its objective, thereby internalizing resource capacity constraints via price signals. This enables each user to proactively balance its QoS (i.e., delay in this work) against network resource consumption when making service migration decisions. To meet practical requirements, adjustable weight parameters will be introduced in subsequent algorithm design, allowing for flexible control over the relative importance of node and link overheads.

Based on direct and dual decompositions, a two-layer hierarchical framework is developed. As shown in Fig. 3, the upper layer serves as the resource coordinator, responsible for node RA and updating the prices of heterogeneous resources, while the lower layer focuses on MD and link RA by solving $\mathbf{P}_5$ with given prices. Compared with $\mathbf{P}_1$, the finer granularity of $\mathbf{P}_5$ leads to lower complexity and substantially reduced state space of the corresponding MDP. With user n 's state given by $S_n^t = (x_n^t, u_n^t)$, the state space size $|S_n^t| = V^2$. Considering the parallel solution capability of the subproblems for each AN, our decomposition reduces the state space from V^{2N} to $N_v \cdot V^2$, where N_v denotes the user number under AN v .

V. CONGESTION-AWARE SCALABLE MIGRATION ALGORITHM

In our layered decomposition framework, the resource prices, i.e., Lagrange multipliers $\lambda^t, \mu^t, \vec{\eta}$ act as interface signals for cross-layer interaction, forming a master-slave closed loop. Built upon this paradigm, we propose the Congestion-Aware Scalable Migration (CASM) algorithm, which is detailed in Alg. 1. Overall, CASM consists of three nested loops.

- The inner loop (loop 3, lines 16-24) corresponds to dual decomposition, where the finest-grained subproblems under a given AN are solved with policy iteration.
- The middle loop (loop 2, lines 9-31) corresponds to direct decomposition, where the per-AN problems are solved in parallel, including MD and node & link RA.
- The outer loop (loop 1, lines 6-34) focuses on link congestion alleviation by evaluating current optimization results and adjusting link weights.

The CASM algorithm runs in discrete time slots, with the entire three-loop nested iterative process executed on a per-time-slot basis. With the decompositions in Section IV, node RA on each AN v is addressed by solving $\mathbf{P}_2(\mathbf{C}, \mathbf{W})$ in parallel, as shown in line 13 of Alg. 1. Then, the node resource prices λ^t, μ^t on each AN are updated following the subgradient method for dual decomposition [34].

$$\lambda^{t, n_2} = \lambda^{t, n_2-1} + \alpha \cdot \left(\sum_{n \in N_v} C_n^t - \bar{C}_v \right)^+ \quad (38)$$

$$\mu^{t, n_2} = \mu^{t, n_2-1} + \alpha \cdot \left(\sum_{n \in N_v} W_n^t - \bar{W}_v \right)^+ \quad (39)$$

where α is the updating rate. These updated resource price signals are then passed to the inner loop to guide the MDP-based service migration, which is elaborated in the following subsection.

A. MDP-based Service Migration

Within the inner loop, the finest-grained $\mathbf{P}_5(\mathbf{R})$ for each single user n is transformed into an MDP, defined as follows.

(1) **State**: the state in time slot t is defined as $S_n^t = (u_n^t, x_n^t)$, including user n 's location u_n^t , and x_n^t is the AN hosting its service instance.

(2) **Action**: As the optimization variable of $\mathbf{P}_5$, R is converted into the action $A_n^t = (x_n^t, k_n^t)$, where k_n^t denotes the backhaul data rate for user n . Here, the continuous interval $[R_n^l, R_n^u]$ in (34) is discretized into K levels, forming the feasible data rate set K_n with $k_n^t \in K_n$. Note that actions are performed at the start of each time slot, whereas user mobility, characterized by the transition probability matrix P_M , may occur at any arbitrary moment. The state transitions are depicted in Fig. 4, where the intermediate state (u_n^{t-1}, x_n^t) is defined as $\hat{s}_n^t$.

(3) **Reward**: the reward function RW is derived from $\mathbf{P}_5$'s objective (33). With the definition of A_n^t , we substitute variables $x = x_n^t$ and $k = k_n^t$ for R in (33), and rewrite g_n as:

$$\begin{aligned}
g_n^t(x, k) &= \xi_n(T_n^s(t) + T_n^c(t) + T_n^m(t)) \\
&+ \lambda^t C_n^t + \mu^t W_n^t + \sum_{e \in E} \eta_e^t \cdot I_{\{x=v\}} k P(i_n^t, e)
\end{aligned} \quad (40)$$

Algorithm 1: Congestion-Aware Scalable Migration (CASM) algorithm

```

1 for  $t = 2, \dots, T$  do
2   initialize parameters:  $C, W, R, \lambda, \mu, \gamma$  ;
3   update routing matrix:  $P = \text{ShortPath}(G, w_e^t)$  ;
4    $n_1, n_2, n_3 \leftarrow 0$  ;
5   // Loop 1: Congestion-aware outer loop
6   while  $n_1 \leq N_1^{\max}$  do
7      $n_1 \leftarrow n_1 + 1$  ;
8     // Loop 2: middle loop for solving  $P_1$ 
9     while  $n_2 \leq N_2^{\max}$  do
10       $n_2 \leftarrow n_2 + 1$  ;
11      divide user subsets  $N_v, v \in V$  ;
12      for  $v \in V$ , in parallel do
13        solve  $\mathbf{P}_2(\mathbf{C}, \mathbf{W})$  on AN  $v$  ;
14        update  $\lambda$  and  $\mu$  by (38), (39) ;
15        // Loop 3: Inner loop for migration decision
16        and link RA on AN  $v$ 
17        while  $n_3 \leq N_3^{\max}$  do
18           $n_3 \leftarrow n_3 + 1$  ;
19          for  $n \in N_v$  do
20            calculate reward  $RW$  by (41), (42) ;
21            update  $\text{policy} = \text{PIter}(P_M, RW)$  ;
22            get action  $A_{n,n_3}^t = \text{policy}(S_n^t)$  ;
23            update  $\bar{\eta}$  by (45) ;
24            if  $\Delta g_n^{n_3} < \epsilon_3$  then
25              break ;
26          for  $n \in N_v$  do
27             $A_{n,n_2}^t \leftarrow A_{n,n_3}^t, R_{n,n_2}^t \leftarrow R_{n,n_3}^t$  ;
28            estimate policy  $EP_{n,v}^{t,n_2}$  by (46) ;
29            estimate load  $EL_{e,v}^{t,n_2}$  for each  $v$  by (47) ;
30            allocate subnet capacities  $\overline{SR}_{e,v}^{t,n_2}$  by (48) ;
31            if  $\Delta C^{n_2} < \epsilon_2 \ \& \ \Delta W^{n_2} < \epsilon_2 \ \& \ \Delta X^{n_2} = 0$  then
32              break ;
33          Evaluate current optimization results:
34           $[CG, \gamma_e, w_e^t, E_{vio}, R] = \text{Eval}(C, W, R)$  ;
35          if  $\Delta CG^{n_1} < \epsilon_1 \ \& \ E_{vio} = \phi$  then
36            break ;
37 Output:  $C, W, R$ 

```

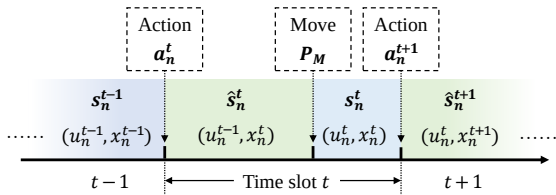


Fig. 4. The state transitions: action a_n^t at the beginning of time slot t causes $s_n^{t-1} \rightarrow s_n^t$, and user movement within time slot t causes $s_n^t \rightarrow s_n^{t+1}$.

where $n \in N_v, t \in T$ and $i_n^t = \Pi(x_n^{t-1}, x)$. Then, the reward of action A_n^t , denoted as $RW(S_n^{t-1}, A_n^t)$, is defined as the reduction in $\mathbf{P}_5$'s objective caused by A_n^t compared with taking no action.

$$RW(S_n^{t-1}, A_n^t) = g_n^t \Big|_{x=x_n^t, k=k_n^t} - g_n^t \Big|_{x=x_n^{t-1}, k=k_n^{t-1}} \quad (41)$$

Subsequently, to make the MDP model congestion-aware and priority-aware, three key parameters ($\omega, \kappa_n, \gamma_e$) are introduced to modify g_n^t (40). The modified objective is formulated to adapt migration decisions to the link congestion status and user priority differentiation, which is given by:

$$g_n^t(x, k, \omega, \kappa_n, \gamma_e) = \xi_n(T_n^s(t) + T_n^c(t) + T_n^m(t)) + \omega_N \cdot NO_n^t + \omega_L \cdot LO_n^t \quad (42)$$

$$NO_n^t = \lambda^t C_n^t + \mu^t W_n^t \quad (43)$$

$$LO_n^t = \kappa_n \sum_{e \in E} \gamma_e \eta_e^t \cdot I_{\{x=v\}} k P(i_n^t, e) \quad (44)$$

First, two weight coefficients ω_N, ω_L are introduced to balance the emphasis on node and link overheads, enabling flexible adjustment of the relative importance of heterogeneous resources. Compared with ω_N , a larger ω_L indicates that CASM places greater emphasis on alleviating link congestion. The relative magnitudes of the two parameters can be configured according to practical deployment scenarios, i.e., whether node or link resources act as the bottleneck that limits the network capacity. Second, we introduce the user weight $\kappa_n = (1 + \xi_n)^{-1}$ to link overhead LO_n^t , which is negatively correlated with the user priority. This design introduces priority-awareness to CASM by making the service migration of low-priority users more constrained by its link overhead. Third, a link-specific weight γ_e is incorporated to revise the link resource price η_e^t , where γ_e is positively correlated with the real-time congestion degree of link e . γ_e is updated along with the outer loop, as elaborated in Section V-C. This induces the MDP model to avoid utilizing already congested links for service migration, thus enabling load-balancing and alleviating congestion on highly demanded links.

With the definitions above, $\mathbf{P}_5$ for each user $n \in N_v$ is converted to a low-complexity MDP and solved with policy iteration, as shown in lines 18-21 of Alg. 1. Then, the link prices η_e^t are updated as below, following the same scheme as λ^t and μ^t .

$$\eta_e^{t,n_2} = \eta_e^{t,n_2-1} + \alpha \left(\sum_{n \in N_v} R_n^t(v) P(i_n^t, e) - \bar{R}_e^t(v) \right) \quad \forall e \in E \quad (45)$$

Finally, as shown in line 23, the inner loop stops when the change $\Delta g_n^{n_3} = |\sum_{n \in N_v} (g_n^{t,n_3} - g_n^{t,n_3-1})|$ falls below a threshold ϵ_3 .

B. Subnet Resource Allocation

As shown in lines 25-29 of Alg. 1, when the inner loop converges, the algorithm estimates future policies based on previous actions, and allocates link resource capacities for each virtual subnet. This process is summarized as $A_n^t \rightarrow EP_{n,v}^t \rightarrow EL_{e,v}^t \rightarrow \overline{SR}_{e,v}^t$, and the key variables are derived as follows.

$$EP_{n,v}^{t,n_2} = \sum_{z=1}^{\min\{Z, n_2\}} q_p^z \cdot A_n^{t,n_2-z}(v) / \sum_{z=1}^Z q_p^z \quad (46)$$

$$EL_{e,v}^{t,n_2} = \sum_{n \in N} EP_{n,v}^{t,n_2} \cdot R_n^{t,n_2} \cdot P(i_n^t, e) \quad (47)$$

$$\overline{SR}_{e,v}^{t,n_2} = \frac{(1 + \delta) \bar{R}_e EL_{e,v}^{t,n_2}}{\sum_{v \in V} EL_{e,v}^{t,n_2}} \quad (48)$$

where $v \in V$, $n \in N$, and $e \in E$. First, we define the estimated policy EP . At the n_2 -th iteration of the middle loop, $EP_{n,v}^{t,n_2} \in [0, 1]$ denotes the probability for service n to migrate to AN v . As shown in (46), EP is obtained via exponential smoothing over the last Z actions, with q_p serving as the attenuation factor, and $A_n^{t,n_2}(v) = 1$ only if $x_n^{t,n_2} = v$. Second, the estimated load on link e originating from user subset N_v , denoted as $EL_{e,v}^{t,n_2}$, is calculated as (47) based on EP and routing matrix P . With these estimated loads, link capacity $\bar{R}_e$ is partitioned among V virtual subnets in proportion to their estimated loads, as shown in (48). Note that the inaccuracy in load estimation may lead to resource insufficiency in underestimated subnets and waste in overestimated ones. To compensate for such inaccuracy and more efficiently exploit the resource on each link, a positive margin δ is introduced for resource over-provisioning. However, under heavy-load conditions, such over-provisioning may cause the actual load L_e^t on certain links to exceed their capacity $\bar{R}_e$. To enforce link capacity constraints, a heuristic throttling strategy is proposed, which is elaborated in Section V-C.

After subnet resource allocation, as shown in line 30, the middle loop stops if the changes in C and W fall below threshold ϵ_2 , and the migration decision X remains unchanged.

C. Congestion-Aware Evaluation

Upon convergence of the middle loop for solving the joint problem $\mathbf{P}_1$, as depicted in line 32 of Alg. 1, the optimization results (C, W, R) are evaluated based on their influence on link congestion. The evaluation is specified in Function 1. First, for each backhaul link $e \in E$, we calculate its load L_e^t with current optimization results, and evaluate its congestion degree measured by resource utilization $\theta_e = L_e^t / \bar{R}_e$, which is formulated as the following piecewise function.

$$Con(\theta_e) = \begin{cases} 1, & \text{if } 0 \leq \theta_e \leq \theta_1 \\ 1 + \epsilon_0, & \text{if } \theta_1 < \theta_e \leq \theta_2 \\ 2 + \epsilon_0, & \text{if } \theta_2 < \theta_e \leq 1 \end{cases} \quad (49)$$

where $0 < \theta_1 < \theta_2 < 1$ are adjustable thresholds, and ϵ_0 is a positive constant. Then, CASM achieves congestion awareness through two schemes. (1) Path selection: The routing matrix P is updated at the beginning of each time slot by calculating weighted shortest paths for each SDP $i \in I_G$ (line 3 of Alg. 1), where the congestion degree $Con(\theta_e)$ is taken as link weight w_e^t . (2) Congestion-based pricing: As presented in line 5 of Function 1, the congested link set E_{con} is derived, where ρ_c is the congestion threshold. Links in E_{con} are penalized by raising their coefficient γ_e (line 8), while non-congested links get moderately relaxed (line 10), where q_c and q_v are adjustable parameters within $[0, 1]$. The update of γ_e guides the MDP to avoid overloaded links, forming a distributed congestion pricing mechanism derived from dual decomposition principles, where local migration decisions are aligned with global load balancing across the backhaul network.

The remaining module in Function 1 is heuristic rate throttling. To avoid link capacity constraint violations caused by over-provisioning in subnet resource allocation, a lightweight heuristic is adopted as shown in lines 12–25 of Function 1. First,

```

1 Function 1 Eval( $C, W, R$ )
2 // (a) Congestion-aware link weight update
3 calculate link load  $L_e^t$  with  $C, W, R$ , then calculate
    $CG = \sum_{e \in E} Con(\theta_e)$ 
4 update link weights:  $w_e^t = Con(\theta_e)$ 
5 get congested link set  $E_{con} = \{e \in E | \theta_e > \rho_c\}$ 
6 for  $e \in E$  do
7   if  $e \in E_{con}$  then
8      $\gamma_e^{n_1} = \gamma_e^{n_1-1} \cdot [1 + q_c(\theta_e - \rho_c)]$ 
9   else
10     $\gamma_e^{n_1} = \gamma_e^{n_1-1} \cdot q_v$ 
11 // (b) Heuristic throttling
12 check link constraints and get violating link set
    $E_{vio} = \{e \in E | L_e^t > \bar{R}_e\}$ 
13 if  $E_{vio} \neq \emptyset$  &  $n_1 \leq N_1^{max} - 2$  then
14   for  $e \in E_{vio}$  do
15     get user set  $N_e = \{n \in N | P(i_n^t, e) = 1\}$ 
16     calculate  $\Delta_e = L_e^t - \bar{R}_e$ 
17     if  $\Delta_e > 0$  then
18       group the users in  $N_e$  by ascending priority:
          $[N_e^1, \dots, N_e^S] = SortP(N_e)$ 
19       for  $s = 1$  to  $S$  do
20         sort users in  $N_e^s$  by descending data rate:
            $N_e^s \leftarrow SortR(N_e^s)$ 
21         for  $n \in N_e^s$  do
22           disable migration of service  $n$ :
              $R_n^t \leftarrow 0, \Delta_e \leftarrow \Delta_e - R_n^t$ 
23           if  $\Delta_e \leq 0$  then
24             return to line 15, check next link
25   update  $L_e^t, CG, w_e^t$ 

```

Output: $CG, \gamma_e, w_e^t, E_{vio}, R$

the violating link set $E_{vio} = \{e \in E | L_e^t > \bar{R}_e\}$ is identified. As the outer-loop index n_1 nears the maximum number of iterations, for each link $e \in E_{vio}$, we determine its associated user set $N_e = \{n \in N | P(i_n^t, e) = 1\}$, and its required load reduction $\Delta_e = L_e^t - \bar{R}_e$. Then, rate throttling is performed with the *Min Priority & Max Reducible First* principle. Specifically, N_e is grouped by ascending priority (*Min Priority First*). Starting from the lowest-priority group, users in each group are sorted by descending data rate (*Max Reducible First*). Service migration of these users is disabled sequentially until $\Delta_e \leq 0$. This heuristic throttling is capable of eliminating constraint violations while inflicting the least performance degradation on high-priority services.

Finally, as shown in line 33 of Alg. 1, the outer loop stops when the change in overall congestion $\Delta CG^{n_1} = CG^{n_1} - CG^{n_1-1}$ falls below threshold ϵ_1 , enabling control of global network congestion.

VI. EXPERIMENTAL EVALUATION

In this section, simulation results are presented to evaluate the CASM algorithm. First, the scalability of our decomposition

framework is verified by comparing the running time of CASM under various configurations. Second, the delay-congestion trade-off is validated, together with the performance of CASM under different migration demands. Finally, the superior performance of CASM in low-connectivity topologies is verified by benchmarking against baseline approaches.

A. Experimental Setup

Table I shows the configurations of system parameters, including the network-related and user service-related parameters. In the default setting, we adopt a 25-node mesh topology as the backhaul network G . As observed from Table I, $I_n^t \gg D_n^t$, implying that backhaul link overhead is dominated by service migration instead of remote communication. Then, by setting $\bar{R}_e$ to a random distribution ranging from 400 to 800 Mbps, we simulated a backhaul network with constrained link resources relative to the migration data size I_n^t and rate k_n^t . User mobility is modeled as a random walk, where each user moves to an adjacent AN within 2 hops with probability p_0 in each time slot, and p_0 is set to 0.2 for the default case. For emulating diverse user priority levels, the user set N is classified into four groups with priorities $\xi_n \in \{1, 2, 3, 4\}$, and the corresponding proportions are 40%, 30%, 20%, 10%, respectively. Besides, the parameter settings involved in the CASM algorithm are elaborated as follows. By default, the discretized data rate set $K_n = \{50, 100, 150, 200\}$ (Mbps) for each user n . ω_N is set to 10, and $\omega_L = 20$ by default. Set thresholds $\epsilon_1 = 5$, $\epsilon_2 = 0.01$, $\epsilon_3 = 0.1$, $\theta_1 = 0.5$, $\rho_c = \theta_2 = 0.75$, and updating rates $\alpha = 0.1$, $q_c = 0.5$, $q_v = 0.95$. As for the over-provisioning in subnet resource allocation, δ is set to 0.2 by default.

As stated previously, CASM is a unified framework comprising the joint optimization of Migration Decision (MD), Path Selection (PS), and Data Rate (DR) allocation. To validate the effectiveness of PS and DR in reducing user delay and network congestion, we adopt the following three baselines.

- MDPS: Derived from CASM by removing DR through fixing $k_n^t = 100$ Mbps for all users, while retaining PS by updating the routing matrix. This algorithm corresponds to the routing-only migration approach in [12].
- MDO (MD Only): Further simplified from MDPS by fixing the routing matrix and holding $\gamma_e = 1$, thus removing congestion-aware price adjustment. This algorithm corresponds to the decomposition-based approach in [9], where link congestion is not considered in the design.
- NM (Never Migrate): No service migration is performed regardless of user mobility, this algorithm provides the performance lower bound for CASM.

B. Simulation Results

(1) Scalability Analysis

To verify the scalability of the proposed two-layer decomposition framework, the average per-slot running time (RT) of CASM is evaluated, with the results depicted in Fig. 5 and Fig. 6. Theoretically, two key factors impact RT . First, the execution time of inner loops is determined by the state and action space sizes of the MDP for $\mathbf{P}_5$, which is dominated

TABLE I
SYSTEM CONFIGURATIONS

Parameter	Value
Time slot number T	20
Network scale V	25 (default)
User number N	80 (default)
User mobility probability p_0	0.2 (default)
User priority ξ_n	1,2,3,4
Backhaul link capacity $\bar{R}_e$	[400, 800] Mbps
AN computation capacity $\bar{C}_v$	8 Mcycles
AN bandwidth capacity $\bar{W}_v$	10 Mbps
Channel condition $SNR(u_n^t)$	[15, 20] dB
Computation task size ϕ_n^t	[1, 2] Mcycles
Remote communication data size D_n^t	[1, 2] MB
Migration data size I_n^t	[90, 110] MB

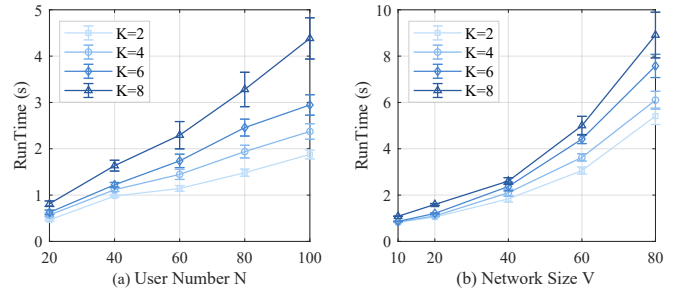


Fig. 5. Average per-slot running time (RT) of CASM with different (a) user number N , (b) network scale V . The variation in RT stems from changes in the state and action spaces of the MDP.

by V , N , and K , the number of discrete data rate levels. Specifically, network topologies with different scales V are generated using the Watts-Strogatz small-world model [38]. The second factor is the number of outer-layer iterations, determined by link congestion CG and the number of link constraint violations, i.e., $|E_{vio}|$. Accordingly, we select distinct p_0 and I_n^t to emulate diverse migration demands corresponding to different congestion degrees, and set $\delta = 0.1, 0.2, 0.3$ to generate varying degrees of link constraint violations. The presented results are averaged across 50 independent repeated experiments.

First, as shown in Fig. 5(a), RT exhibits a linear growth as N increases from 20 to 100. Meanwhile, the results in Fig. 5(b) demonstrate that RT is proportional to V^2 . Besides, RT increases linearly with K , since the action space size $|A_n^t| = KV$ is proportional to K . Furthermore, a larger K leads to more significant fluctuations in RT . These results are consistent with the analysis of the MDP state space size $|S_n^t| = N_v V^2$ in Section IV, which validates the effectiveness of the proposed two-layer decomposition framework and its applicability in high user density scenarios.

Second, Fig. 6 illustrates the growth of RT with migration demands. Horizontally, RT rises noticeably as we increase p_0 and I_n^t . Rising p_0 from 0.05 to 0.5 yields a 2.21 $\times$ increase in RT , while enlarging I_n^t from 20 to 200 MB brings a 3.86 $\times$ growth. Vertically, a larger δ also elevates RT . The reason is that greater over-provisioning of link resources during subnet

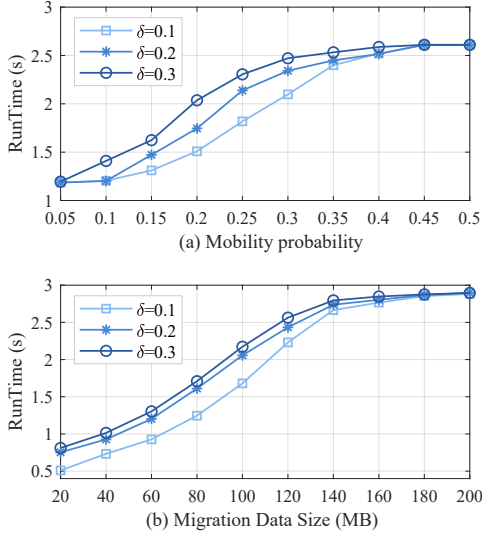


Fig. 6. Average per-slot running time (RT) of CASM with different (a) user mobility probability p_0 , (b) migration data size I_n^t . The variation in RT stems from changes in the number of outer-loop iterations.

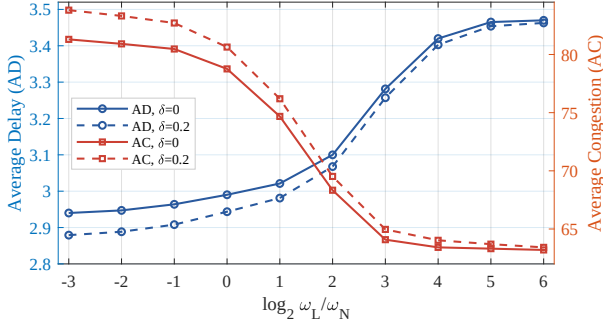


Fig. 7. The average user delay (AD) and average congestion (AC) versus ω_L/ω_N , illustrating the trade-off between user delay and network congestion.

allocation leads to more constraint violations and hence more outer-loop iterations. Finally, it can be seen that when $p_0 > 0.4$ or $I_n^t > 160$ MB, RT stops growing because the number of outer-loop iterations has reached its upper bound N_1^{max} .

(2) Delay-Congestion Trade-off

Fig. 7 illustrates the trade-off between Average user delay (AD) and average congestion (AC) defined as follows.

$$AD = \frac{1}{NT} \sum_{t=1}^T \sum_{n \in N} \xi_n T_n(t), \quad AC = \frac{1}{T} \sum_{t=1}^T \sum_{e \in E} Con(\theta_e^t)$$

Fixing $\omega_N = 10$, the weight ratio $\log_2(\omega_L/\omega_N)$ increases from -3 to 6, representing rising link resource prices and growing emphasis on congestion control. Intuitively, AD rises while AC decreases with the increase of ω_L/ω_N , indicating that the alleviation of link congestion comes at the cost of user delay, since higher link resource prices drive services to suspend migration or migrate at lower data rates. This verifies the trade-off between AD and AC in resource-constrained networks. In practical scenarios, the optimal ω_L/ω_N ratio can be determined according to Service Level Agreements (SLAs) on user delay, thus achieving the best congestion control performance.

Furthermore, AD and AC under $\delta = 0$ and $\delta = 0.2$ are compared. As observed from the dashed curves in Fig. 7,

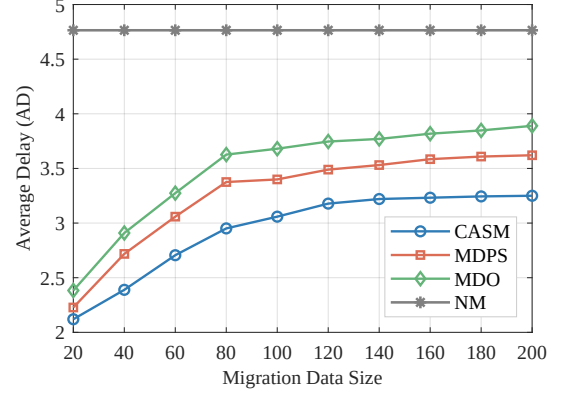


Fig. 8. Performance comparison of CASM and three baselines: the average user delay AD as a function of migration data size I_n^t .

introducing resource over-provisioning can reduce AD and increase AC , and such variation diminishes as ω_L grows. The reason is that resource over-provisioning in subnet partitioning enables more efficient utilization of link resource but aggravates network congestion. By raising the weight of link overhead in (42), a larger ω_L suppresses congestion but also weakens the delay performance gain brought by the over-provisioning of link resources.

(3) Delay Comparison

As depicted in Fig. 8, with p_0 fixed to 0.2, the AD of CASM, MDPS, and MDO rises substantially with the migration data size I_n^t , while NM keeps a constant AD . Since NM does not migrate services, its delay is affected by p_0 rather than I_n^t . Thanks to the joint optimization of MD, PS, and DR, CASM yields the minimum AD , and the performance gap between CASM and baseline schemes expands as I_n^t increases. Specifically, when $I_n^t = 100$ MB, the AD of CASM is 82% of MDO and 90% of MDPS. In addition, the AD growth of all three algorithms saturates when $I_n^t > 120$ MB, as the backhaul links become fully loaded, and the AD ceiling is governed by user mobility probability.

Thus, as a key parameter affecting the algorithm performance boundary, the effect of p_0 on AD is evaluated in Fig. 9. Evidently, the AD of all four algorithms grows with p_0 , as stronger user mobility leads to more frequent service migration requirements. CASM yields the lowest AD when $p_0 \geq 0.2$, while it performs similarly to MDPS and MDO when $p_0 \leq 0.15$. The reason is that low migration demand leads to low link utilization, making the performance benefit of CASM's congestion control unremarkable.

To analyze the underlying reason for the performance gain of CASM, Fig. 10 compares the migration rate (MR) under varying p_0 , with its definition given below.

$$MR = \frac{1}{T} \sum_{t=2}^T \frac{\sum_{n \in N} I\{x_n^t \neq x_n^{t-1}\}}{\sum_{n \in N} I\{u_n^t \neq u_n^{t-1}\}}$$

When $p_0 \leq 0.15$, all three algorithms show 100% MR because backhaul link resources are abundant to accommodate all migration requests. When $p_0 \geq 0.2$, MR decreases with p_0 , and CASM maintains a higher MR compared with two baselines. Therefore, for higher migration demand, the

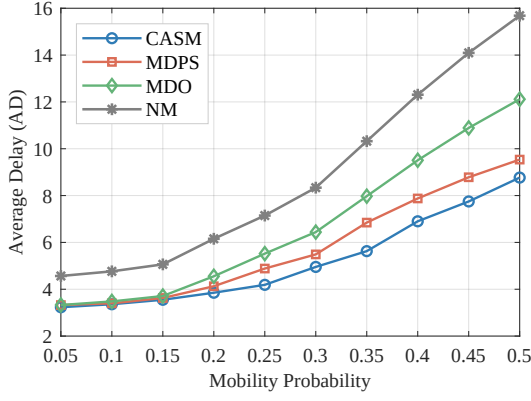


Fig. 9. Performance comparison of CASM and three baselines: the average user delay AD as a function of user mobility probability p_0 .

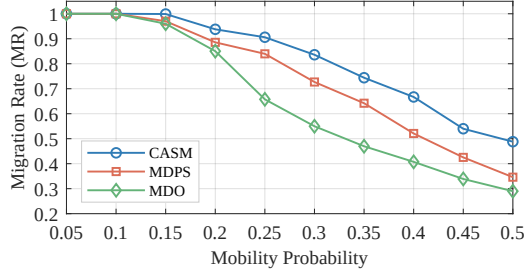


Fig. 10. Migration rate MR as a function of user mobility probability p_0 .

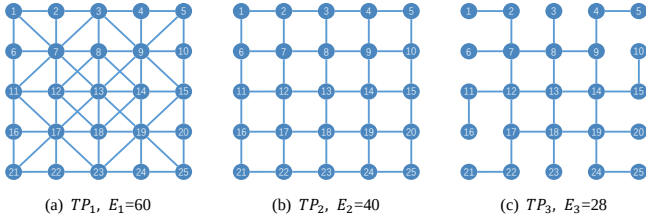


Fig. 11. Three 25-node network topologies with decreasing connectivity from TP_1 to TP_3 .

congestion-aware mechanism of CASM can enable more service migrations under limited link resources, thus reducing user delay.

(4) Performance Under Varying Connectivity

To evaluate the impact of network connectivity on service migration performance, we employ three 25-node topologies as depicted in Fig. 11. TP_1 features the highest connectivity level with $E_1 = 40$, corresponding to backhaul networks in urban areas. Conversely, TP_3 is a sparsely connected topology with $E_3 = 28$, where outer nodes exhibit even lower connectivity, standing for networks in remote areas or emergency disaster relief scenarios. In Fig. 12, besides comparing AD of CASM, MDPS, and MDO, we also evaluate the congestion degree by congested link rate, defined as $CLR = E^{-1} \sum_{e \in E} f(\theta_e)$, where f is an activation function defined as follows.

$$f(\theta) = \begin{cases} 0, & \text{if } 0 \leq \theta \leq 0.5 \\ 2\theta - 1, & \text{if } 0.5 < \theta \leq 1 \end{cases}$$

As shown in Fig. 12(a), with the network connectivity decreasing from TP_1 to TP_3 , the average delay of all three

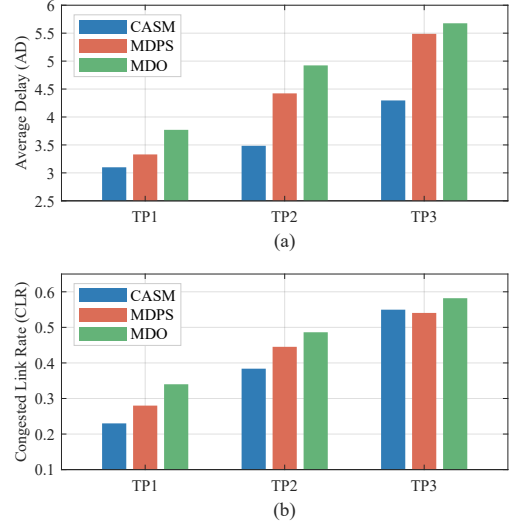


Fig. 12. Performance comparison of (a) average user delay AD , (b) congestion link rate CLR , under different network topologies.

algorithms increases significantly, while CASM achieves the lowest AD , and this gap widens as connectivity decreases. In the sparsely connected network TP_3 , CASM's AD is 75% of MDO's. Compared with TP_1 , the AD growth rates of CASM, MDPS, and MDO in TP_3 are 38.6%, 50.5%, and 64.8%, respectively, which proves that CASM's delay performance is less affected by network connectivity, thanks to the integration of data rate allocation for each migrating service.

As illustrated in Fig. 12(b), CLR rises as network connectivity decreases, because the same migration demand has to be borne by fewer links, leading to increased link congestion. In TP_1 and TP_2 , CASM's CLR is much lower than that of the baseline algorithms, confirming its strong congestion suppression ability. In TP_3 , the CLR of the three algorithms shows no obvious gap, but CASM still achieves a significantly lower AD than the baselines. The reason lies in that sparse topologies provide very limited alternative paths between nodes. In contrast to schemes that solely rely on path selection, CASM incorporates data rate allocation, which allows it to perform fine-grained link resource optimization according to user priorities, thus attaining a lower average delay under the same level of link congestion.

VII. CONCLUSION AND FUTURE WORK

In resource-constrained WANs, service migration involves a fundamental trade-off between user delay and backhaul link congestion, along with severe scalability challenges. This work introduced a novel subnet partitioning method to enable per-node decomposition, and designed the congestion-aware MDP-based algorithm, which jointly optimizes migration decision, path selection, and data rate allocation. Experiments confirmed that CASM exhibits favorable scalability and reduces average user delay by up to 25%, with the gain most pronounced in low-connectivity networks. Our future work will focus on more refined subnet resource allocation schemes, and investigate proactive migration strategies incorporating mobility prediction to enhance performance on both QoS and congestion control.

REFERENCES

- [1] F. S. Abkenar, P. Ramezani, S. Iranmanesh, S. Murali, D. Chulerttiyawong, X. Wan, A. Jamalipour, and R. Raad, "A survey on mobility of edge computing networks in iot: State-of-the-art, architectures, and challenges," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2329–2365, 2022.
- [2] L. Zhang, P. Yu, C. Li, C. Tan, X. Chen, and D. Liao, "Trust-aware rapid emergency service recovery for low-altitude intelligent networking via transformer-enhanced reinforcement learning," in *2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pp. 2381–2386, IEEE, 2025.
- [3] G. Sun, X. Zhan, and J. Such, "Building better ai agents: A provocation on the utilisation of persona in llm-based conversational agents," in *Proceedings of the 6th ACM Conference on Conversational User Interfaces*, pp. 1–6, 2024.
- [4] T. Das, V. Sridharan, and M. Gurusamy, "A survey on controller placement in sdn," *IEEE communications surveys & tutorials*, vol. 22, no. 1, pp. 472–503, 2019.
- [5] F. Guo and M. Peng, "Efficient mobility management in mobile edge computing networks: Joint handover and service migration," *IEEE Internet of Things Journal*, vol. 10, no. 20, pp. 18237–18247, 2023.
- [6] K. Tamada, Y. Kawamoto, and N. Kato, "Bandwidth usage reduction by traffic prediction using transfer learning in satellite communication systems," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 5, pp. 7459–7463, 2023.
- [7] S. Jaiswal, K. Jain, Y. Simmhan, A. Parayil, A. Mallick, R. Wang, R. S. Amant, C. Bansal, V. Ruhle, A. Kulkarni, *et al.*, "Sageserve: Optimizing llm serving on cloud data centers with forecast aware auto-scaling," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 9, no. 3, pp. 1–24, 2025.
- [8] D. Xia, J. Wan, P. Xu, and J. Tan, "Deep reinforcement learning-based qos optimization for software-defined factory heterogeneous networks," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 4058–4068, 2022.
- [9] Z. Li, C. Jiang, and J. Lu, "Distributed service migration in satellite mobile edge computing," in *2021 IEEE Global Communications Conference (GLOBECOM)*, pp. 1–6, IEEE, 2021.
- [10] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 1, pp. 856–868, 2018.
- [11] H. Liu, X. Long, Z. Li, S. Long, R. Ran, and H.-M. Wang, "Joint optimization of request assignment and computing resource allocation in multi-access edge computing," *IEEE Transactions on Services Computing*, vol. 16, no. 2, pp. 1254–1267, 2022.
- [12] J. Xu, X. Ma, A. Zhou, Q. Duan, and S. Wang, "Path selection for seamless service migration in vehicular edge computing," *IEEE Internet of Things Journal*, vol. 7, no. 9, pp. 9040–9049, 2020.
- [13] M. R. Anwar, S. Wang, M. F. Akram, S. Raza, and S. Mahmood, "5g-enabled mec: A distributed traffic steering for seamless service migration of internet of vehicles," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 648–661, 2021.
- [14] H. Xiao, Z. Hu, A. Xu, and M. Zheng, "Cost and qos-aware service migration in vehicular edge computing networks with limited backhaul capacity," in *2024 IEEE Smart World Congress (SWC)*, pp. 369–376, IEEE, 2024.
- [15] Cloud-Native SD-WAN Project, "Cloud-Native SD-WAN (CN-WAN)." GitHub. [Online]. Available: <https://github.com/CloudNativeSDWAN>. Accessed: 2026-04-24.
- [16] N. Toumi, M. Bagaa, and A. Ksentini, "Machine learning for service migration: A survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 3, pp. 1991–2020, 2023.
- [17] Q. Yuan, J. Li, H. Zhou, T. Lin, G. Luo, and X. Shen, "A joint service migration and mobility optimization approach for vehicular edge computing," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 8, pp. 9041–9052, 2020.
- [18] Y. Peng, X. Tang, Y. Zhou, J. Li, Y. Qi, L. Liu, and H. Lin, "Computing and communication cost-aware service migration enabled by transfer reinforcement learning for dynamic vehicular edge computing networks," *IEEE Transactions on Mobile Computing*, vol. 23, no. 1, pp. 257–269, 2022.
- [19] Z. Gao, L. Yang, and Y. Dai, "Vrccs-ac: Reinforcement learning for service migration in vehicular edge computing systems," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 4436–4450, 2024.
- [20] S. Khamari, A. Rachedi, T. Ahmed, and M. Mosbah, "Adaptive deep reinforcement learning approach for service migration in mec-enabled vehicular networks," in *2023 IEEE Symposium on Computers and Communications (ISCC)*, pp. 1075–1079, IEEE, 2023.
- [21] Z. Chen, S. Huang, G. Min, Z. Ning, J. Li, and Y. Zhang, "Mobility-aware seamless service migration and resource allocation in multi-edge iov systems," *IEEE Transactions on Mobile Computing*, 2025.
- [22] S. Wang, Y. Guo, N. Zhang, P. Yang, A. Zhou, and X. Shen, "Delay-aware microservice coordination in mobile edge computing: A reinforcement learning approach," *IEEE Transactions on Mobile Computing*, vol. 20, no. 3, pp. 939–951, 2019.
- [23] J. Wang, J. Hu, G. Min, Q. Ni, and T. El-Ghazawi, "Online service migration in mobile edge with incomplete system information: A deep recurrent actor-critic learning approach," *IEEE Transactions on Mobile Computing*, vol. 22, no. 11, pp. 6663–6675, 2022.
- [24] L. Ale, N. Zhang, X. Fang, X. Chen, S. Wu, and L. Li, "Delay-aware and energy-efficient computation offloading in mobile-edge computing using deep reinforcement learning," *IEEE Transactions on Cognitive Communications and Networking*, vol. 7, no. 3, pp. 881–892, 2021.
- [25] H. Zhou, K. Jiang, X. Liu, X. Li, and V. C. Leung, "Deep reinforcement learning for energy-efficient computation offloading in mobile-edge computing," *IEEE Internet of Things Journal*, vol. 9, no. 2, pp. 1517–1530, 2021.
- [26] T. Ouyang, Z. Zhou, and X. Chen, "Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing," *IEEE Journal on Selected Areas in Communications*, vol. 36, no. 10, pp. 2333–2345, 2018.
- [27] I. Labriji, F. Meneghello, D. Cecchinato, S. Sesia, E. Perraud, E. C. Strinati, and M. Rossi, "Mobility aware and dynamic migration of mec services for the internet of vehicles," *IEEE Transactions on Network and Service Management*, vol. 18, no. 1, pp. 570–584, 2021.
- [28] X. Chen, Y. Bi, X. Chen, H. Zhao, N. Cheng, F. Li, and W. Cheng, "Dynamic service migration and request routing for microservice in connected vehicles," *IEEE Internet of Things Journal*, vol. 9, no. 15, pp. 13126–13143, 2022.
- [29] C. Wang, J. Peng, L. Cai, H. Peng, W. Liu, X. Gu, and Z. Huang, "Ai-enabled spatial-temporal mobility awareness service migration for connected vehicles," *IEEE Transactions on Mobile Computing*, vol. 23, no. 4, pp. 3274–3290, 2023.
- [30] H. Zhou, Q. Li, N. Shi, Y. Chai, Y. Zhang, and Y. Teng, "Energy-efficient collaborative ai service migration and resource allocation based on traffic prediction in 6g networks," in *2025 IEEE International Conference on Communications Workshops (ICC Workshops)*, pp. 1682–1687, IEEE, 2025.
- [31] S. Huang, H. Yao, T. Mai, Z. Wang, and C. Jin, "Uninterrupted service in emergency networks: A multi-strategy service migration approach," *IEEE Transactions on Vehicular Technology*, 2025.
- [32] Y. Sun, S. Zhou, and J. Xu, "Emm: Energy-aware mobility management for mobile edge computing in ultra dense networks," *IEEE Journal on Selected Areas in Communications*, vol. 35, no. 11, pp. 2637–2646, 2017.
- [33] X. Zhou, S. Ge, T. Qiu, K. Li, and M. Atiquzzaman, "Energy-efficient service migration for multi-user heterogeneous dense cellular networks," *IEEE Transactions on Mobile Computing*, vol. 22, no. 2, pp. 890–905, 2021.
- [34] M. Chiang, S. H. Low, A. R. Calderbank, and J. C. Doyle, "Layering as optimization decomposition: A mathematical theory of network architectures," *Proceedings of the IEEE*, vol. 95, no. 1, pp. 255–312, 2007.
- [35] B. Cao, H. Ye, J. Liu, B. Tang, Z. Tao, and S. Deng, "Smart: Cost-aware service migration path selection based on deep reinforcement learning," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 9, pp. 12421–12436, 2024.
- [36] Y. Liu, G. Feng, Z. Chen, S. Qin, and G. Zhao, "Network function migration in softwarization based networks with mobile edge computing," in *ICC 2020-2020 IEEE International Conference on Communications (ICC)*, pp. 1–6, IEEE, 2020.
- [37] M. A. Jasim, N. Siasi, M. Almalag, V. Honary, and A. Aldalbahi, "Asynchronous coarse-grained load migration scheme for iot applications in fog networks," in *2021 IEEE Global Conference on Artificial Intelligence and Internet of Things (GCAIoT)*, pp. 82–87, IEEE, 2021.
- [38] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world' networks," *Nature*, vol. 393, no. 6684, pp. 440–442, 1998.